Beam analysis using MATLAB

Exercise A

Stiffness method and analytical solution

This guide explains how the MATLAB solution is built and reproduces its result by hand. The direct stiffness calculation and the slope-deflection calculation use the same Euler-Bernoulli beam theory; their agreement checks the assembly, boundary conditions, and signs.

The fixed-end support reaction moments are 18.500 kN m counterclockwise at support 1 and 20.375 kN m clockwise at support 3.

Determine the supports 1 and 3. EI is constant. Assume joint 2 is a roller

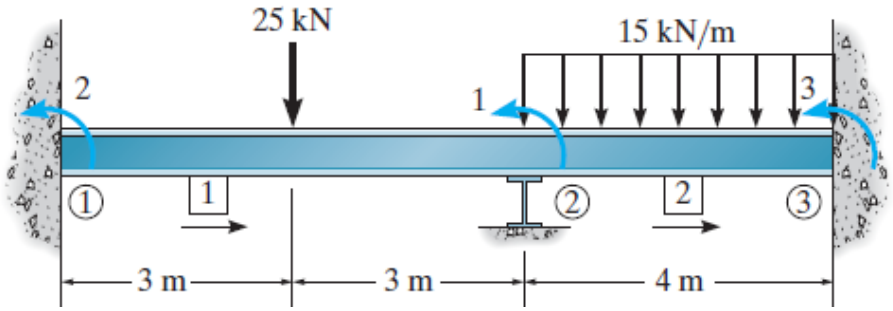


Figure 1. Circled numbers identify physical supports; other blue labels in the textbook illustration are not the global degree-of-freedom numbering used in the code.

Use kN, m, and radians. The model assumes a straight prismatic beam, linear elastic behaviour, small displacements and rotations, and negligible shear deformation. Both ends are fixed; the intermediate roller allows rotation while maintaining continuity of the beam.

1 Define the model and its degrees of freedom

The code creates one beam element per span. Each node has a vertical displacement and a bending rotation. There are six global degree-of-freedom positions, but the boundary conditions leave only the rotation at node 2 unknown.

$$\mathbf{d} = [v_1, \theta_1, v_2, \theta_2, v_3, \theta_3]^T$$

Index	Physical variable	Condition
1	v_1	0
2	θ_1	0
3	v_2	0
4	θ_2	unknown
5	v_3	0
6	θ_3	0

MATLAB representation

```
xNode = [0 6 10];
connectivity = [1 2; 2 3];
prescribedDOF = [1 2 3 5 6];
freeDOF = 4;
d = zeros(6,1);
d(3) = 0;
```

xNode stores coordinates, and each row of connectivity stores the node pair of one element. The index rule 2*i-1 selects v at node i; 2*i selects its rotation. Therefore element 1 maps to [1 2 3 4], and element 2 maps to [3 4 5 6]. The common entries 3 and 4 make displacement and rotation continuous at the roller.

```
i = connectivity(e,1);
j = connectivity(e,2);
elementDOF(e,:) = [2*i-1, 2*i, 2*j-1, 2*j];
```

Sign convention: vertical forces and displacements are positive upward; nodal moments and rotations are positive counterclockwise. Internal bending moment is positive in sagging. A downward load is consequently negative. A negative prescribed displacement represents settlement.

A roller cannot supply an external reaction couple. It does not release bending continuity within a continuous beam, so the internal moment at node 2 is generally nonzero. An internal hinge would require a different model.

2 Derive the element stiffness and load vectors

For an element running from node i to node j, define the local coordinate from left to right. The element displacement vector is $[v_i, \theta_i, v_j, \theta_j]^T$. Because both elements have the same global orientation, their local and global bending axes coincide.

$$\mathbf{k}_e = \frac{EI}{L^3}\begin{bmatrix} 12 & 6L & -12 & 6L \\ 6L & 4L^2 & -6L & 2L^2 \\ -12 & -6L & 12 & -6L \\ 6L & 2L^2 & -6L & 4L^2 \end{bmatrix}$$

This matrix follows from the strain energy of an Euler-Bernoulli beam. Cubic Hermite interpolation relates displacement within the element to the four nodal variables. Differentiating twice gives curvature; integrating curvature energy gives the matrix.

$$\mathbf{k}_e = \int_0^L E\,I\,\mathbf{B}^T\mathbf{B}\,dx, \qquad \mathbf{B} = \frac{d^2\mathbf{N}}{dx^2}$$

Consistent loading

A span load contributes equivalent nodal forces and equivalent nodal moments. These are chosen to preserve virtual work for the interpolation functions. The equivalent nodal vector is not the same as the fixed-end member action vector: the latter has the opposite sign.

$$\mathbf{f}_e = \int_0^L \mathbf{N}^T\,q\,dx + \mathbf{N}(a)^T P, \qquad \mathbf{s}_e = \mathbf{k}_e\mathbf{d}_e - \mathbf{f}_e$$

$$\mathbf{f}_{e,q} = q\begin{bmatrix} L/2 \\ L^2/12 \\ L/2 \\ -L^2/12 \end{bmatrix}$$

For a concentrated force at ξ = a/L, the four Hermite functions are evaluated at the force location. At the middle of the span, they reduce to the weights shown below. The rotational weights have units of length and turn the force into nodal moments.

$$\mathbf{N}(a)^T = \begin{bmatrix} 1-3\xi^2+2\xi^3 \\ L(\xi-2\xi^2+\xi^3) \\ 3\xi^2-2\xi^3 \\ L(-\xi^2+\xi^3) \end{bmatrix}, \qquad \mathbf{f}_{e,P}\big|_{a=L/2} = P\begin{bmatrix} 1/2 \\ L/8 \\ 1/2 \\ -L/8 \end{bmatrix}$$

3 Evaluate the span loads and assemble the arrays

Insert the actual length and the signed physical load into each element formula. In every load vector below, entries 1 and 3 are in kN, while entries 2 and 4 are in kN m.

$$\mathbf{f}_1 = -25\begin{bmatrix} 1/2 \\ 6/8 \\ 1/2 \\ -6/8 \end{bmatrix} = \begin{bmatrix} -12.5 \\ -18.75 \\ -12.5 \\ 18.75 \end{bmatrix}$$

$$\mathbf{f}_2 = -15\begin{bmatrix} 4/2 \\ 4^2/12 \\ 4/2 \\ -4^2/12 \end{bmatrix} = \begin{bmatrix} -30 \\ -20 \\ -30 \\ 20 \end{bmatrix}$$

Using only two vertical forces of 12.5 kN for the point load would discard the nodal moments. That would change the free-rotation equation and produce an incorrect support-moment distribution.

What the loop does

```
K = zeros(nDOF);
F = zeros(nDOF,1);
elementK = zeros(4,4,nElement);
elementLoad = zeros(4,nElement);
L = zeros(nElement,1);
% For each element: determine L, form ke and fe, then assemble.
elementK(:,:,e) = ke;
elementLoad(:,e) = fe;
dof = elementDOF(e,:);
K(dof,dof) = K(dof,dof) + ke;
F(dof) = F(dof) + fe;
```

The storage and assembly statements after the comment belong inside the element loop. ke and fe are formed earlier in that loop from the formulas on the previous page. The stored matrices are retained for recovering member end actions after the displacement solution.

K(dof,dof) selects the four global rows and four global columns associated with this element. The plus sign adds contributions rather than overwriting them. At node 2, stiffness and equivalent loads from both spans enter the same global entries. elementK(:,:,e) uses a third array dimension to store one 4 by 4 matrix per element.

In Exercise A the point-load branch first calculates ξ = a/L, then N, and adds P*N to fe. In Exercise A that branch is skipped because P(e) equals zero. All load signs remain tied to the upward-positive convention.

4 Read the global system and solve its free row

The assembled matrix below follows the degree-of-freedom order [v_1, θ_1, v_2, θ_2, v_3, θ_3]. It is K divided by the common EI; the script uses the reference EI = 1.

Row / column	1	2	3	4	5	6
1	0.055556	0.166667	-0.055556	0.166667	0.000000	0.000000
2	0.166667	0.666667	-0.166667	0.333333	0.000000	0.000000
3	-0.055556	-0.166667	0.243056	0.208333	-0.187500	0.375000
4	0.166667	0.333333	0.208333	1.666667	-0.375000	0.500000
5	0.000000	0.000000	-0.187500	-0.375000	0.187500	-0.375000
6	0.000000	0.000000	0.375000	0.500000	-0.375000	1.000000

The table is rounded for reading; the code retains full precision. Matrix entries have mixed units because translation and rotation occupy different columns and force and moment occupy different rows. Symmetry is a useful assembly check.

$$\mathbf{Kd} = \mathbf{F} + \mathbf{R}, \qquad \mathbf{F} = \begin{bmatrix} -12.5 \\ -18.75 \\ -42.5 \\ -1.25 \\ -30 \\ 20 \end{bmatrix}$$

Separate free entries f from prescribed entries c. Only row 4 is needed to find the free rotation. Rows at prescribed degrees of freedom are retained for subsequent reaction recovery.

$$\mathbf{K}_{ff}\mathbf{d}_f = \mathbf{F}_f - \mathbf{K}_{fc}\mathbf{d}_c$$

$$\left(\frac{4EI}{6} + \frac{4EI}{4}\right)\theta_2 = -1.25 \quad \Rightarrow \quad \theta_2 = -\frac{0.75}{EI}$$

Here d_c is zero. The unknown common EI cancels when member actions are recovered, because the rotation scales as 1/EI. Raw reference rotations and deflections are not physical numerical predictions. The code consequently plots EI times displacement.

```
Kff = K(freeDOF,freeDOF);
Kfc = K(freeDOF,prescribedDOF);
effectiveLoad = F(freeDOF) - Kfc*d(prescribedDOF);
d(freeDOF) = Kff \ effectiveLoad;
```

Backslash solves the reduced linear system. Explicitly forming inv(Kff) is unnecessary. The original unconstrained K is singular because rigid motions have not yet been removed; the reduced system contains the support constraints.

5 Reproduce the solution analytically

Use slope-deflection to obtain the same end moments without assembling a global matrix. Define M_{ij} as the action applied to the member at its i end, with counterclockwise positive at both ends. Define the chord displacement $\Delta = v_j - v_i$, measured upward. This convention matches endAction in the code.

$$M_{ij} = \frac{2EI}{L}\left(2\theta_i + \theta_j - \frac{3\Delta}{L}\right) + M_{ij}^F$$

$$M_{ji} = \frac{2EI}{L}\left(\theta_i + 2\theta_j - \frac{3\Delta}{L}\right) + M_{ji}^F$$

The fixed-end terms M^f are the actions required when both ends of an element are held at zero displacement and rotation. They equal minus the rotational entries of fe. Set $\theta_1 = \theta_3 = 0$, and enforce $M_{21} + M_{23} = 0$ because there is no externally applied couple at node 2.

Span	Left fixed-end moment	Right fixed-end moment
1	+25 × 6 / 8 = +18.75	-18.75
2	+15 × 4² / 12 = +20	-20

$$M_{12} = \frac{EI}{3}\theta_2 + 18.75, \qquad M_{21} = \frac{2EI}{3}\theta_2 - 18.75$$

$$M_{23} = EI\theta_2 + 20, \qquad M_{32} = \frac{EI}{2}\theta_2 - 20$$

$$\frac{5EI}{3}\theta_2 + 1.25 = 0 \quad \Rightarrow \quad \theta_2 = -\frac{0.75}{EI}$$

$$M_{12} = 18.5, \quad M_{21} = -19.25, \quad M_{23} = 19.25, \quad M_{32} = -20.375$$

All end moments above are in kN m. This hand calculation has the same scalar free-rotation equation as the matrix solution. The agreement checks the fixed-end loads, support settlement signs, and joint equilibrium without relying on the assembly loop.

6 Recover member end actions and support reactions

Insert the solved displacement vector into every element. Recover the element actions by subtracting the equivalent span-load vector. Then assemble the support reactions from Kd − F. A displacement boundary condition creates an unknown reaction; an unconstrained rotation cannot create a support reaction moment.

$$\mathbf{s}_e = \mathbf{k}_e\mathbf{d}_e - \mathbf{f}_e, \qquad \mathbf{R} = \mathbf{K}\mathbf{d} - \mathbf{F}$$

```
R = K*d - F;
for e = 1:nElement
    de = d(elementDOF(e,:));
    endAction(:,e) = elementK(:,:,e)*de - elementLoad(:,e);
end
M_support = [-endAction(2,1); endAction(4,1); endAction(4,2)];
```

Each endAction column contains $[V_i, M_i, V_j, M_j]^T$. The local nodal displacements de are selected from d using elementDOF. The two end forces at node 2 add to its vertical reaction; the two end moments at that node cancel.

The vertical member actions can also be recovered by hand from moment and vertical-force equilibrium of each span. With signed q and P, use the following expressions before adding the member forces at the roller.

$$V_j = -\frac{M_i + M_j + qL^2/2 + Pa}{L}, \qquad V_i = -qL - P - V_j$$

Element	V_i in kN	M_i in kN m	V_j in kN	M_j in kN m
1	12.375000	18.500000	12.625000	-19.250000
2	29.718750	19.250000	30.281250	-20.375000

Support	Vertical reaction in kN	Reaction moment in kN m
1	12.375000	18.500000
2	42.343750	0.000000
3	30.281250	-20.375000

Convert external end actions to the internal diagram

$$M(0) = -M_i, \qquad M(L) = M_j$$

Support	Internal bending moment in kN m
1	-18.500000
2	-19.250000
3	-20.375000

The left support reaction moment and the left internal diagram moment have opposite signs because they act on different free bodies. All three support-section internal moments here are negative, meaning hogging. The right support reaction is clockwise because R(6) is negative.

7 Derive shear force and bending moment between nodes

For a cut at local position x, use equilibrium of the segment to the left of the cut. A point force contributes only after its application point. The step H is zero before the force and one after it; the ramp bracket means max(x − a, 0).

$$V(x) = V_i + qx + PH(x-a), \qquad M(x) = -M_i + V_i x + \frac{qx^2}{2} + P\langle x-a\rangle$$

For span 1, x is measured from support 1. For span 2, use a different local coordinate z = global x − 6 m. These local expressions recover the exact response of the ideal beam under the specified loads.

$$V_1(x) = \begin{cases} 12.375 & 0 < x < 3 \\ -12.625 & 3 < x < 6 \end{cases}$$

$$M_1(x) = -18.5 + 12.375x - 25\langle x-3\rangle, \quad 0 \le x \le 6$$

$$V_2(z) = 29.71875 - 15z, \qquad M_2(z) = -19.25 + 29.71875z - 7.5z^2$$

At the 25 kN force, shear drops by 25 kN and moment remains continuous. At the roller, shear rises from −12.625 to +29.71875 kN, a jump of 42.34375 kN equal to the support reaction.

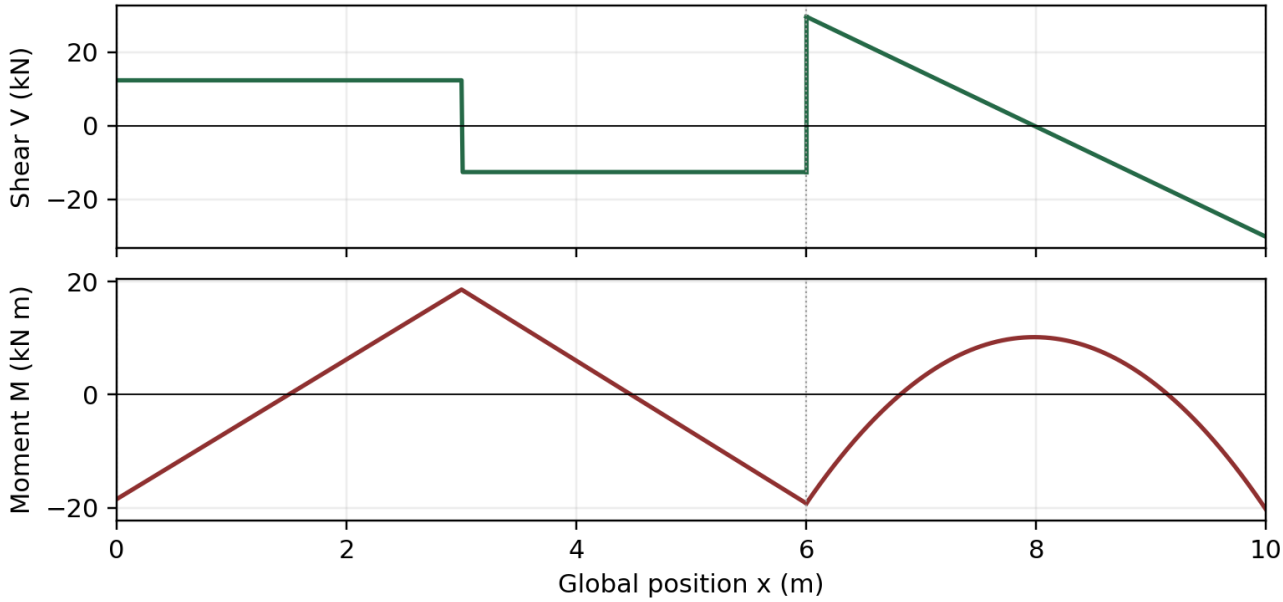


Figure 2. Response calculated from span equilibrium. Moment is positive in sagging; the negative support values indicate hogging.

8 Integrate bending moment to recover deflection

With the chosen signs, EI times the second derivative of displacement equals internal bending moment. Integrate once for rotation and twice for displacement. The known left-end rotation and displacement supply the integration constants.

$$EI\, v''(x) = M(x)$$

$$\theta(x) = \theta_i + \frac{-M_i x + V_i x^2/2 + q x^3/6 + P\langle x-a\rangle^2/2}{EI}$$

$$v(x) = v_i + \theta_i x + \frac{-M_i x^2/2 + V_i x^3/6 + q x^4/24 + P\langle x-a\rangle^3/6}{EI}$$

The MATLAB arrays x, ramp, V, M, and v store values throughout each element. A dot before a power, as in x.^2, means element-by-element exponentiation. This is required because x is a vector. Cell arrays xPlot{e} and vPlot{e} store vectors that can have different lengths for different elements.

```
ramp = max(x-a(e),0);
M = -s(2) + s(1)*x + q(e)*x.^2/2 + P(e)*ramp;
v = de(1) + de(2)*x + ...
    (-s(2)*x.^2/2 + s(1)*x.^3/6 + q(e)*x.^4/24 + ...
     P(e)*ramp.^3/6)/EI;
```

A curve obtained only by joining nodal displacements would miss bending under the span loads. The integration above retains those loads. It is exact for this ideal prismatic beam model, rather than an approximation limited to a cubic curve between solved nodal values.

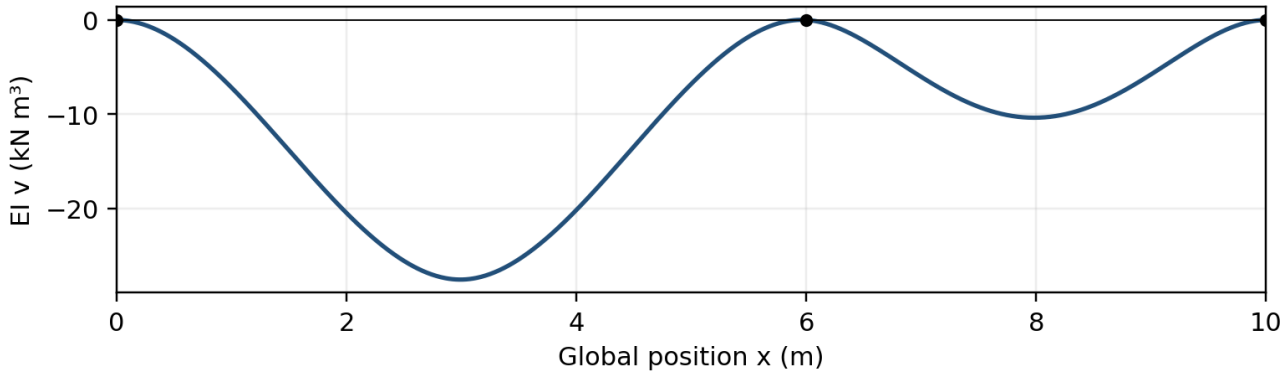


Figure 3. Stiffness-scaled displacement EI v. Divide by the actual EI in kN m² to obtain v in metres.

At each element right end, the recovered displacement and rotation must match de(3) and de(4). The code checks this compatibility. It provides a stronger check than simply plotting a smooth-looking curve.

9 Verify equilibrium and compatibility

A numerical result should satisfy the original physical problem, not just the matrix equations used to generate it. Check whole-beam equilibrium using the actual forces and load resultants. Do not add equivalent nodal loads to those same physical loads, because that counts the loading twice.

$$\sum F_y = 12.375 + 42.34375 + 30.28125 - 25 - 15(4) = 0$$

$$\sum M_1 = 18.5 - 20.375 + 42.34375(6) + 30.28125(10) - 25(3) - 60(8) = 0$$

$$M_{21} + M_{23} = -19.25 + 19.25 = 0$$

The displayed numbers are rounded, so hand substitution may leave a small rounding residual. The script evaluates these checks with full precision and uses numerical tolerances rather than requiring exact floating-point zero.

Checks implemented in the script

Check	What it tests
K equals its transpose	Reciprocity and correct symmetric assembly
Vertical and moment residuals are near zero	Whole-beam force and moment equilibrium
R(freeDOF) is near zero	No reaction couple at an unrestrained rotation
Two member moments at node 2 add to zero	Joint moment equilibrium
Recovered v and theta meet right-end nodal values	Displacement and rotation compatibility
M_support matches referenceMoments	Original exercise agrees with the hand solution

```
assert(norm(R(freeDOF),inf) < 1e-8, ...
    'An unrestrained rotation cannot develop a support reaction moment.');
assert(abs(endAction(4,1)+endAction(2,2)) < 1e-8, ...
    'Member end moments must balance at the unloaded roller joint.');
```

When intentionally changing geometry, loads, or settlement, disable the comparison with the original referenceMoments. Keep the equilibrium and compatibility checks. In Exercise A, changing EI also changes the combined reference answer.

10 Follow the script during the lecture

The lesson source is exercise_A_beam_stiffness_live_ready.m. Run its sections in order. The table relates every stage to a mechanical calculation and the variable that should be inspected.

Stage	Code task	Mechanical meaning or key output
1	Initialise	clear, clc, close all; start a reproducible calculation
2	Define input	xNode, connectivity, q, P, a, EI, settlement
3	Number unknowns	elementDOF maps element entries into d
4	Prescribe supports	freeDOF and prescribedDOF; impose d(3)
5	Allocate arrays	K, F, elementK, elementLoad, L
6	Form and assemble	ke and fe; add them into K and F
7	Inspect full system	Check shared-node entries and signed F
8	Reduce and solve	Kff, Kfc, effectiveLoad; solve theta2
9	Recover actions	R, endAction, and M_support
10	Interpret tables	External reactions versus internal diagram moments
11	Recover fields	xPlot, VPlot, MPlot, vPlot
12	Verify	Force and moment residuals; compatibility checks
13	Plot	Beam schematic and structural response diagrams
14	Experiment	Change inputs and predict the effect before rerunning

Commands worth explaining to students

zeros allocates arrays; size and numel obtain dimensions; colon selects all entries in a row or column; parentheses index numeric arrays; braces extract a vector stored in a cell array; .^ applies a power to each vector entry; backslash solves a system; and assert stops execution when a required check fails.

subplot creates four panels; hold on overlays both spans; quiver draws load arrows; patch draws a support symbol; and grid on helps read the diagrams. The schematic is a drawing, whereas vPlot, VPlot, and MPlot are calculated structural fields.

A final interpretation check

Ask students why the roller has no reaction moment but the internal section moment is nonzero. Then ask why the left support reaction moment and left diagram moment have opposite signs. A correct explanation should identify the free rotation, beam continuity, and the different free bodies used for reactions and internal forces.

Source basis: the supplied exercise diagram and the accompanying MATLAB lesson. Every numerical result in this guide follows from the displayed beam equations and loads. Figures 2 and 3 were calculated from those equations; they are not screenshots of MATLAB execution.